Project 1 – Solve the Robot Warehouse Problem through Search

Due: 23:59 PM, 3/1/2026 | **Total:** 100 Points (including 10pts Acknowledgement)

Objectives

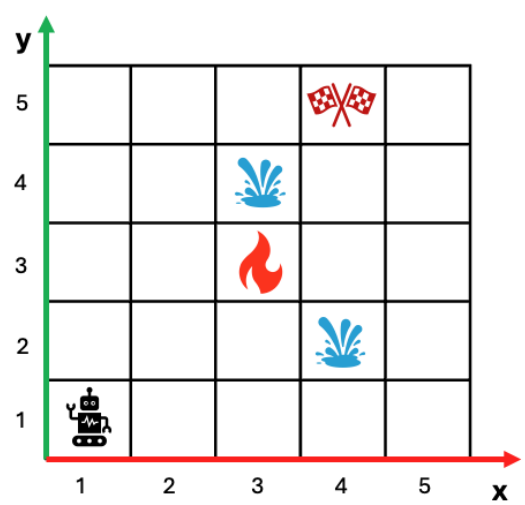
1. **Implementation:** Develop informed search algorithms in Python to find optimal paths in a weighted grid.
2. **Analysis:** Evaluate the performance of different heuristics and weight parameters based on completeness, optimality, time, and space complexity.

Submission Requirements

- **project1.py**: A complete script using the **provided template**.
- **report.pdf**: A written report answering all tasks with rationales and diagrams.
- **Strict Library Constraint**: You are not allowed to import any additional Python libraries, modules, or packages. **The only permitted import is `*import math*`, which is already provided in the template.**
- **Data Structure Requirement:** You must implement your own data structure for the **Frontier** (priority queue logic) and the **Reached** set (lookup table) from scratch using standard Python lists and dictionaries.
- **Your Python script is expected to compile and run without errors and without requiring any additional libraries to be installed on the instructor's or TA's computers.**

Problem Scenario

- **Environment:** A 5x5 grid where (1,1) is the bottom-left and (5,5) is the top-right.
- **Agent:** A robot starting at (1,1) with the goal of reaching (4,5).
- **Direction Costs:**
  - **Cardinal (N, S, E, W):** $action_cost = 1.0$.
  - **Diagonal:** $action_cost = 1.41$ (i.e., $\sqrt{2}$).
- **Fragile Override: Moving into fragile cells overrides the direction cost**: **(3,3)**: $action_cost = 5$, **(4,2)**: $action_cost = 2$, **(3,4)**: $action_cost = 3$.



©2026 Trudi Qi

Task 1: Heuristic Comparison (60 pts)

Implement the **A* Search** algorithm where the evaluation function is $f(n) = g(n) + h(n)$.

Programming (40 pts)

Complete a_star_search(heuristic) to:

1. **Trace Output:** At each step, print the node being expanded and the current frontier.
   - *Search node format:* Parent_State -> Current_State (g=path_cost, f=total_cost).

```
=== A* Search (euclidean) ===
Initial frontier: [None->(1, 1)(0.00, 5.00)]

Step 1
Expanded: None->(1, 1)(0.00, 5.00)
Frontier: [(1, 1)->(2, 2)(1.41, 5.02), (1, 1)->(1, 2)(1.00, 5.24), (1, 1)->(2, 1)(1.00, 5.47)]

Step 2
Expanded: (1, 1)->(2, 2)(1.41, 5.02)
Frontier: [(2, 2)->(2, 3)(2.41, 5.24), (1, 1)->(1, 2)(1.00, 5.24), (1, 1)->(2, 1)(1.00, 5.47), (2, 2)->(3, 2)(2.41, 5.57), (2, 2)->(1, 3
)(2.82, 6.43), (2, 2)->(3, 1)(2.82, 6.94), (2, 2)->(3, 3)(6.41, 8.65)]
```

2. **Results:** Print the final solution path as a sequence of (x, y) coordinates, the total path cost, and the total number of nodes generated.

```
Goal reached.
Path: [(1, 1), (2, 2), (3, 3), (4, 4), (5, 5), (6, 6)]
Total estimated cost (f=g+h) = some number
Nodes expanded =  some number
Nodes generated = some number
```

3. **Heuristic:** Support simply switching the input argument: “manhattan” or “euclidean”:

```python
# Test Case (run twice and compare)
if __name__ == "__main__":
    start = (1, 1)
    goal = (4, 5)

    a_star_search(start, goal, heuristic_type="manhattan", weight=1.0)
    a_star_search(start, goal, heuristic_type="euclidean", weight=1.0)
```

Written Report (20 pts)

Task 1.1 (4 pts): Present the final output for both **Manhattan** and **Euclidean** heuristics.

Task 1.2 (16 pts) Deep Thought Questions:

A. **Path Optimality (4 pts):** Did both heuristics find an optimal solution? Explain why.

B. **Efficiency (4 pts):** Which heuristic is more efficient? Explain why.

C. **Admissibility (8 pts):** For a grid allowing diagonal moves, and assuming there are no fragile cells, are Manhattan and Euclidean **admissible** heuristics, respectively? Explain why or why not based on the definition of a "relaxed problem".

©2026 Trudi Qi

Task 2: Weighted A* Experiments (30 pts)

Experiment with the behavior of A* by **modifying the weight** of the heuristic:

$$f(n) = g(n) + W \cdot h(n)$$

Task 2.1: Heavy Weight (W = 5.0) (18 pts)

A. **Results (5 pts):** Provide the solution path, total cost, and nodes generated for both heuristics using $W = 5.0$.

B. **Comparison (7 pts):** Compare these results to Task 1 ($W = 1.0$). Did the "Heavy Weight" search find the solution faster (fewer nodes)? Was the resulting path still the cheapest possible?

C. **Visualization (6 pts):** Draw the 5x5 grid **for each heuristic**. Plot the $W = 1.0$ path and $W = 5.0$ path in different colors **<u>and</u> explain** how "greedy" weight changes the robot's behavior.

Task 2.2: Zero Weight (W = 0.0) (12 pts)

A. **Algorithm Identification (5 pts):** When $W = 0.0$, $f(n) = g(n)$. Which specific search algorithm are we now using? Provide the solution path and the number of generated nodes.

B. **Optimality Analysis (7 pts):** Compare the number of generated nodes of $W = 0.0$ to your **best heuristic result** from Task 1. Explain the "cost of optimality": Why does this algorithm find the best path but expand so many more nodes?

Acknowledgement & AI Disclosure (10 pts)

To ensure academic accountability, you must provide:

1. **Resources:** List all course materials (slides/page numbers) and websites used.
2. **AI Usage:** Clarify whether (YES or No) you've used AI. If you used AI (e.g., ChatGPT, Gemini):
    - Specify exactly which parts of the code or report the AI assisted with.
    - Explain how you verified the AI's output.

Rubric for Written Report

- **Completeness (25%):** All questions answered and all required program outputs included.
- **Correctness (25%):** Logic follows the rules of A* and the specific grid costs provided.
- **Clarity & Format (25%):** Use of clear headers, readable coordinate sequences, and professional diagrams.
- **Depth of Discussion (25%):** Evidence of thinking beyond the code (e.g., correctly applying terms like "Admissible," "Uniform-Cost," and "Heuristic Relaxation").

©2026 Trudi Qi